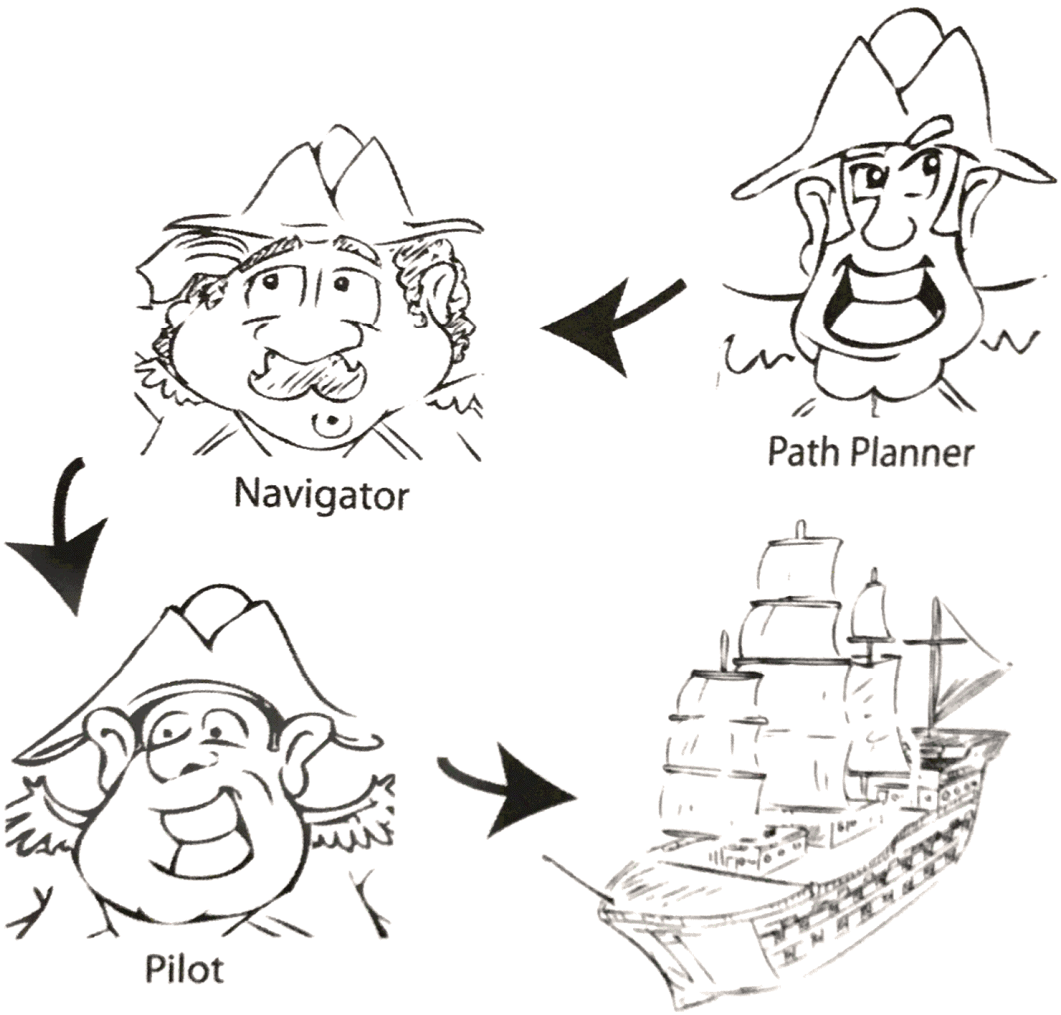
Introduction to Robotics: Planning

Shuai Zhang

Department of Computer Science
University of Central Oklahoma

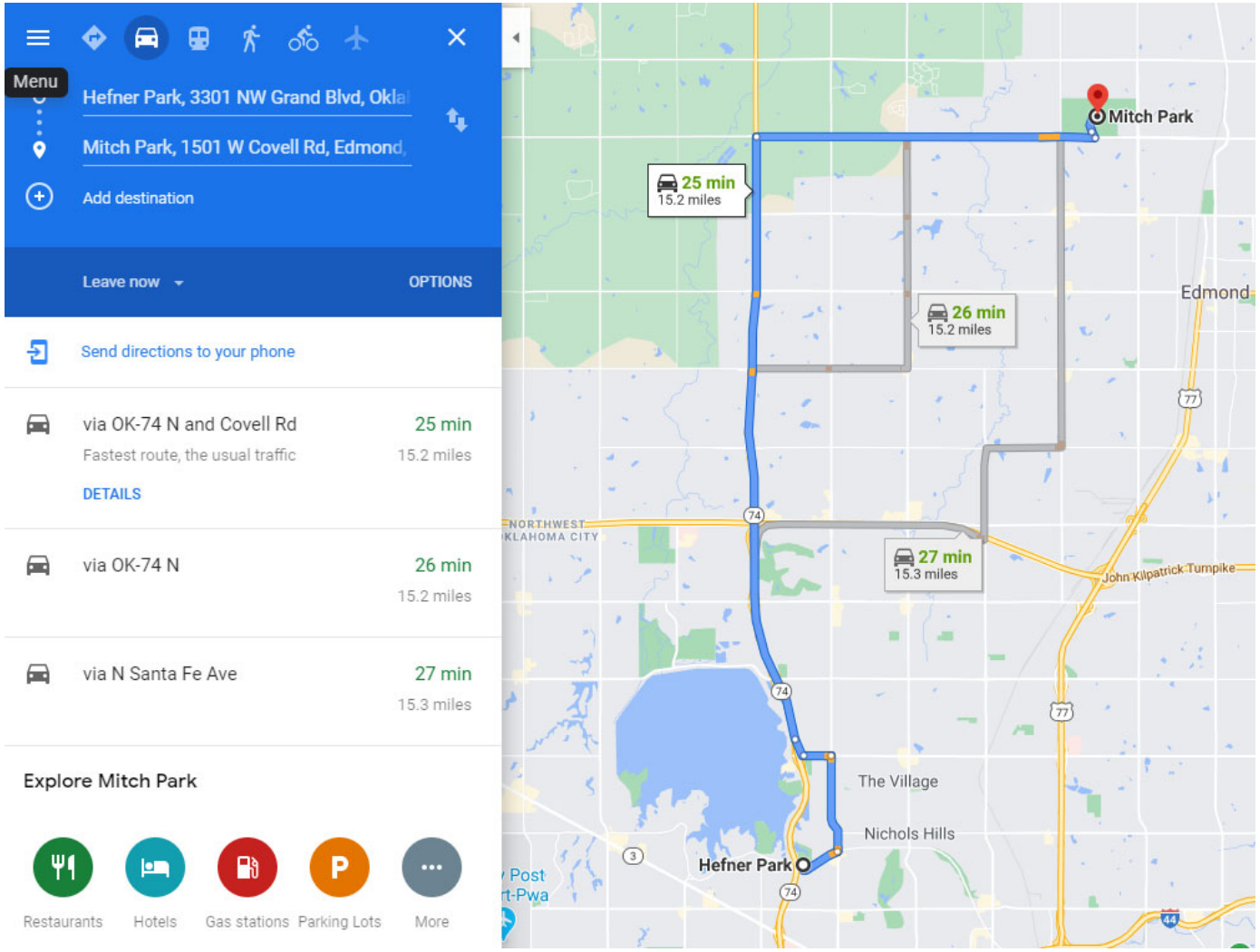
Credit to the book Introduction to Autonomous Mobile Robots, by Roland Siegwart and Illah Nourbakhsh

Roles In Navigation 1



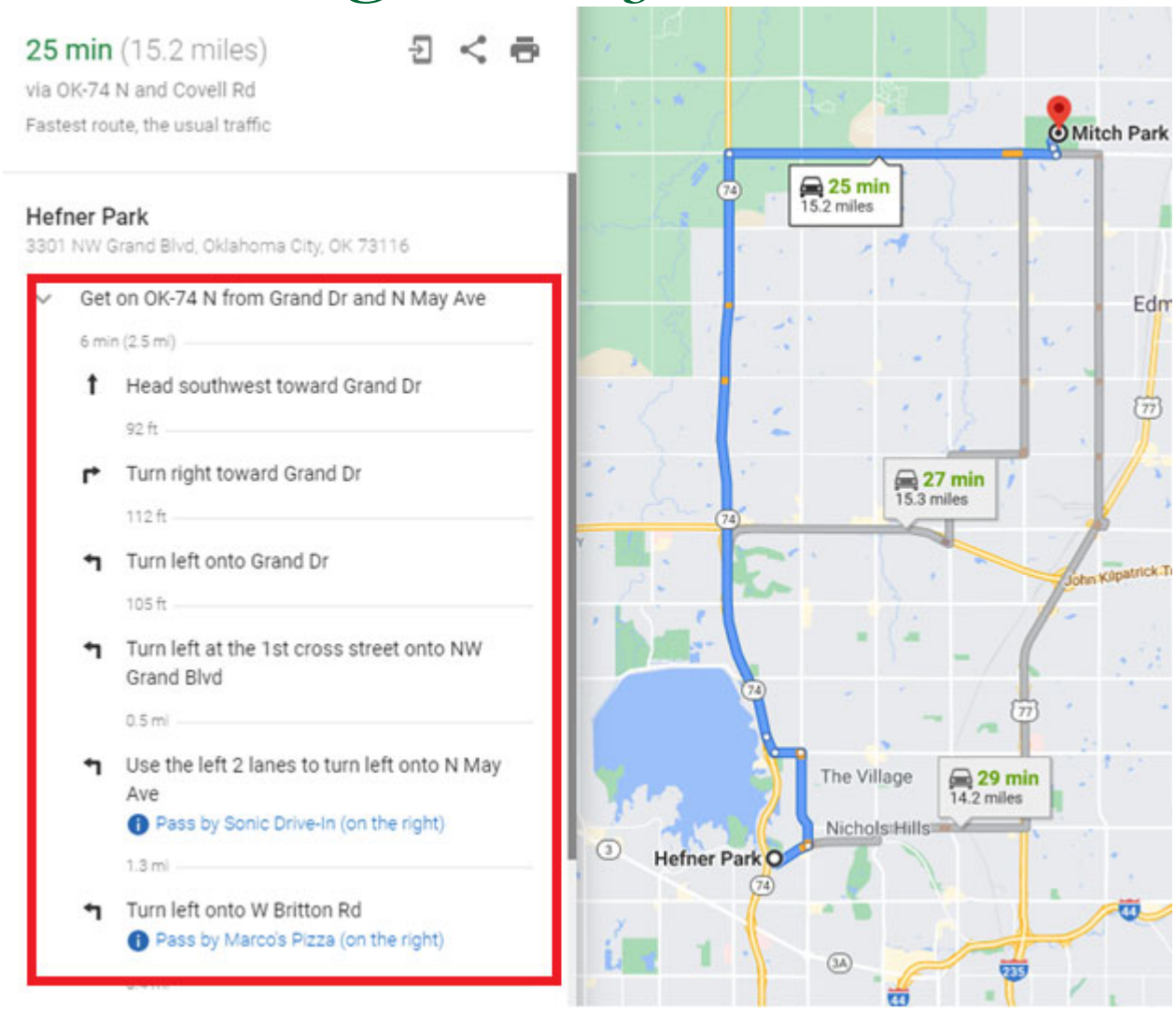
Three roles: layered design

Roles In Navigation 2



Path Planner: multiple paths, select one

Roles In Navigation 3



Navigator: detailed instructions on the path + traffic

Roles In Navigation 4

Pilot: control the steering wheels, brake, etc.

Roles In Navigation 5

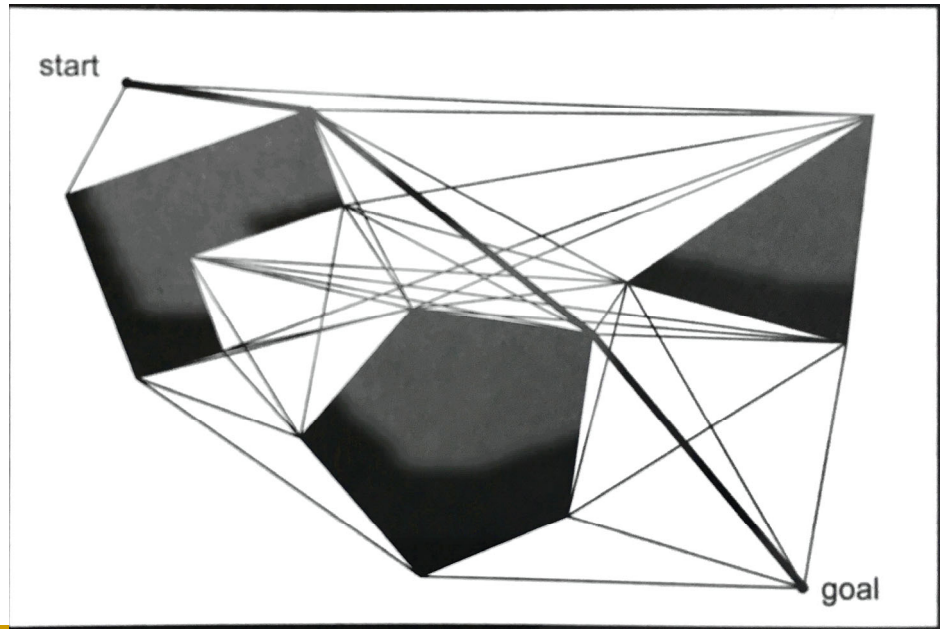
- Kinematics for the pilots: get the right parameters for the robot actuators
  - Otherwise, the robot will go to the wrong location
- Localization and mapping for the navigators: get the right location and update the map based on the new observations
  - Otherwise, the path need to be updated
- Path planning for the path planner: find the suitable path for the robot (avoid heavy traffic area)
  - Otherwise, extra cost for a path

Roles In Navigation 6

- Involved tasks for Path Planner
  - Environment abstraction: trajectory decomposition / segmentation
    - Well-defined start and goal
    - Waypoints
  - Path primitives (line, circle, etc.)
  - Obstacle detection and avoidance
  - Optimization and constraint

Environment Abstraction 1

- The Visibility Graph
  - We can use polygons to configure space obstacles
  - Nodes which are visible from each other are connected by straight-line
    - Edges along each polygon's sides
  - The path planner's task is to find a path from initial point to goal point along the roads defined by the visibility graph

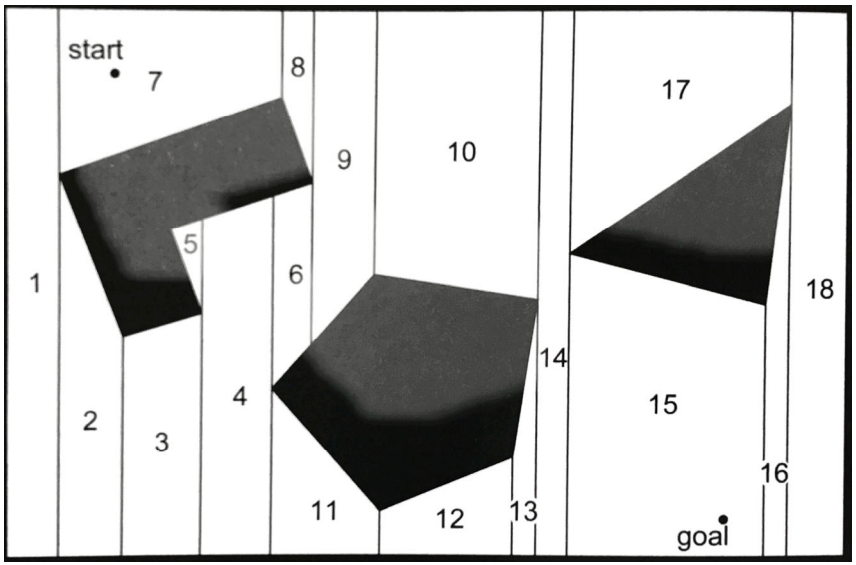


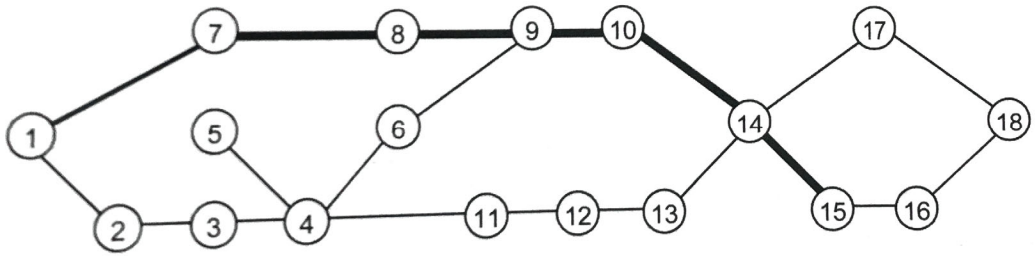
Visibility Graph

https://www.research-collection.ethz.ch/bitstream/handle/20.500.11850/68831/1/eth-4369-01.pdf

Environment Abstraction 2

- Exact cell decomposition
  - Assume the particular position of a robot within each area doesn't matter -- when the precise location is not very important in a rough plan
  - Each cell can then be abstracted as node in a graph model
  - Benefit: **simple** (but **not very accurate**)



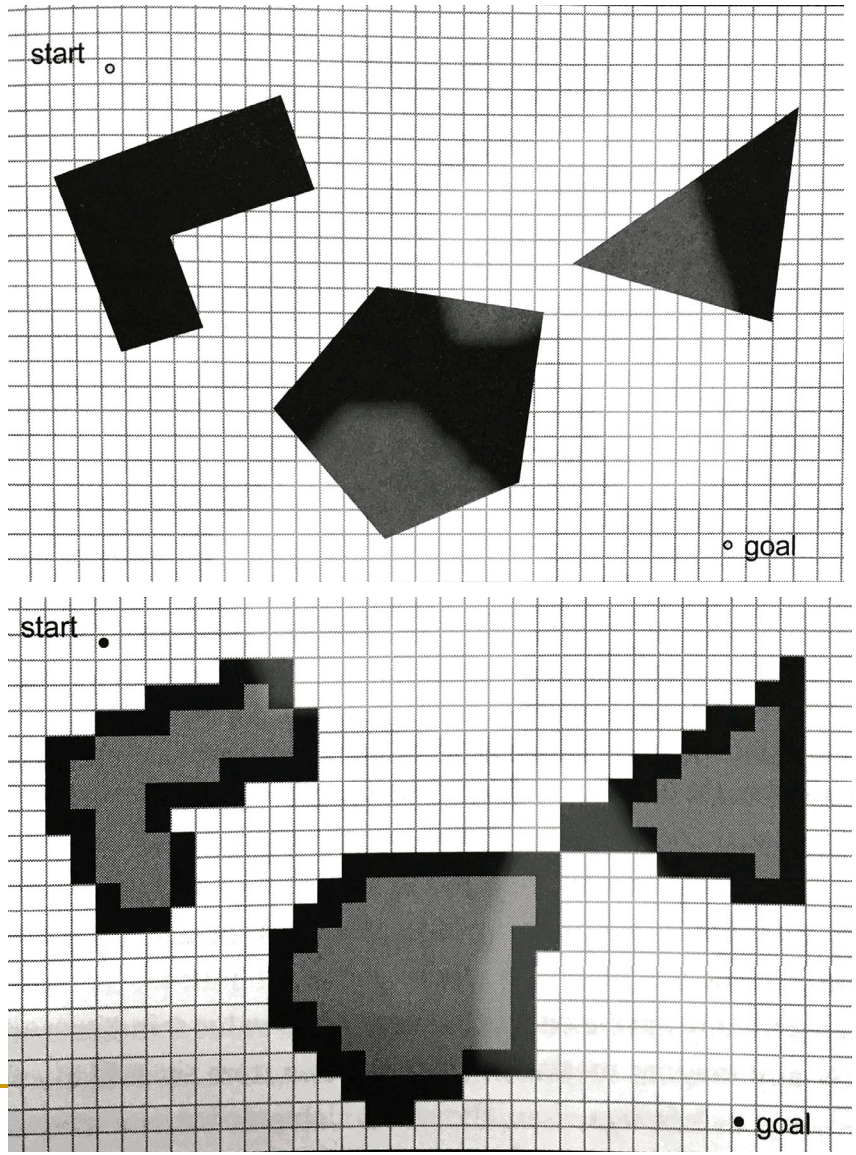


Exact cell decomposition

https://www.research-collection.ethz.ch/bitstream/handle/20.500.11850/68831/1/eth-4369-01.pdf

Environment Abstraction 3

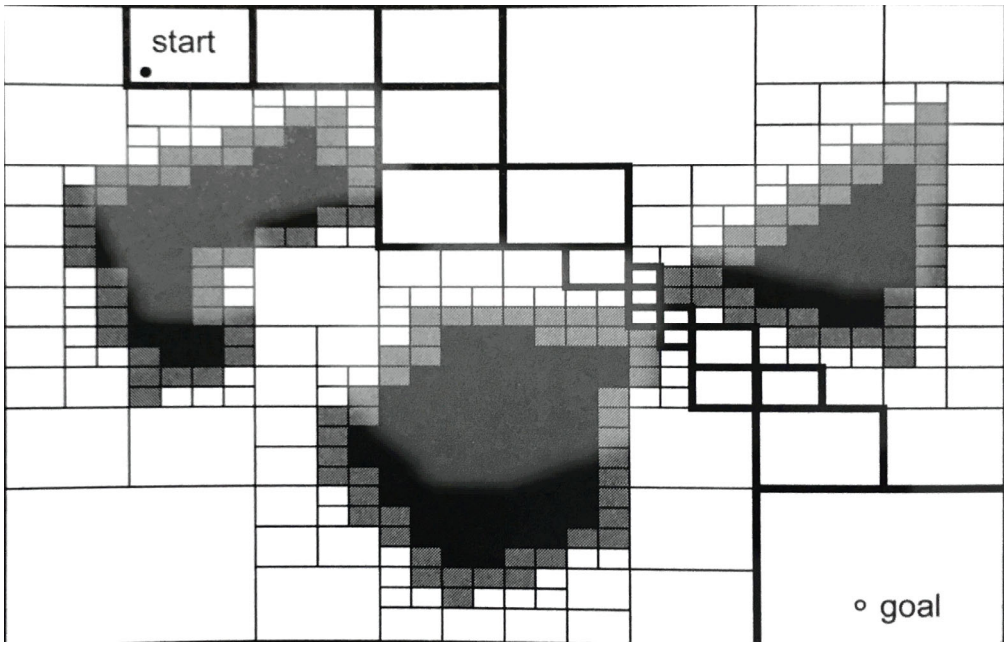
- Fixed decomposition
  - Transform the continuous real environment into a discrete approximation
  - **Easy implementation** + simple + relatively accurate
  - Some path may be lost, e.g. the path between the triangle and the pentagon in the second graph



Fixed decomposition

Environment Abstraction 4

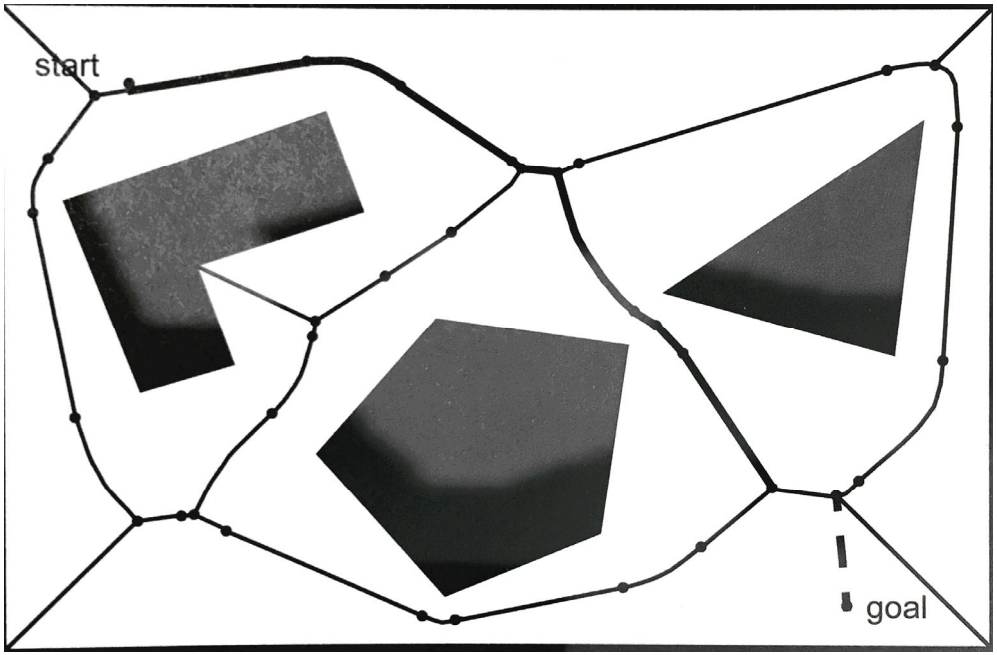
- Adaptive decomposition
  - We can use several pre-defined rectangles to fit into different spaces
    - A larger one can be decomposed into four smaller ones
  - **Adjustable/adaptive accuracy**
    - The path between the triangle and the pentagon can be found
  - How: recursively try a fit from larger ones to smaller ones, to get an accurate fit



Adaptive decomposition

Environment Abstraction 5

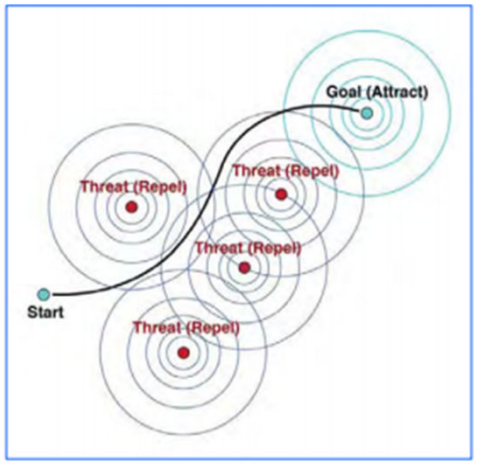
- Voronoi diagram
  - The Voronoi diagram consists of the lines constructed from all points that are **equidistant** from two or more obstacles
  - Find the closest point to the start and goal point
  - Related to sophisticated Computational Geometry
    - We can use tools that are ready

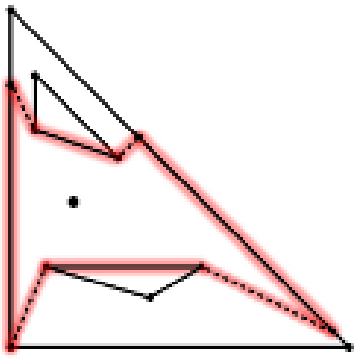


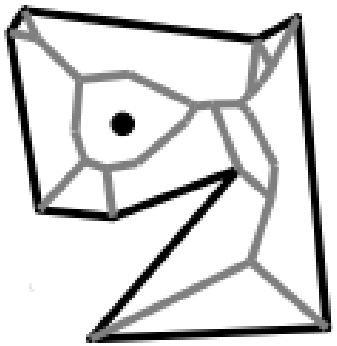
Voronoi diagram

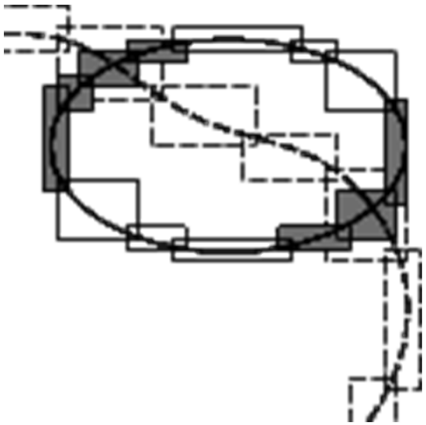
Environment Abstraction 6

- Computational Geometry
  https://www.cgal.org/
  - Segmentation
  - Iso-oriented boxes
  - Voronoi Diagram



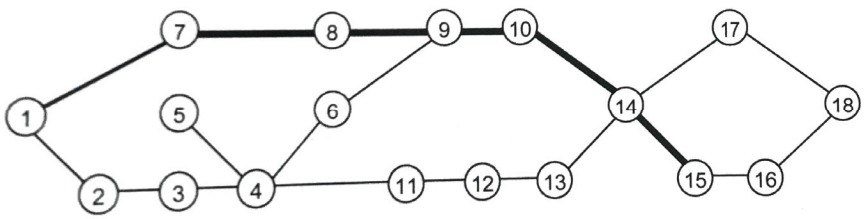


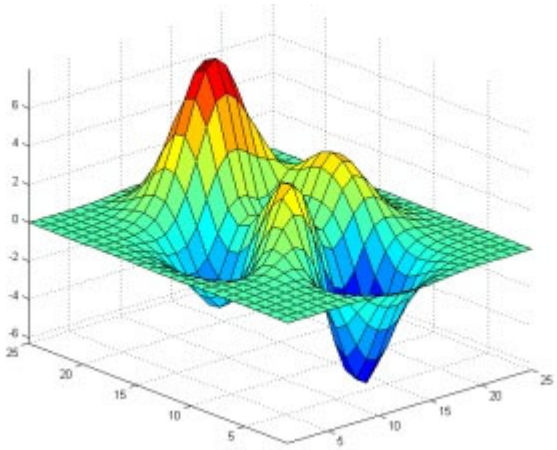




Path Planning Algorithms 1

- After environment abstraction, we will have a graph representation of a math function representation
- Next step is to find an optimal solution
  - Basically this is a search problem





Path Planning Algorithms 2

- Common used algorithms
  - Depth First Search / Breadth First Search
  - Dijkstra's algorithm
  - A* search
  - Simulated Anneal Algorithm
  - Genetic Algorithm
- You can learn them in Data Structures, Artificial Intelligence, etc.

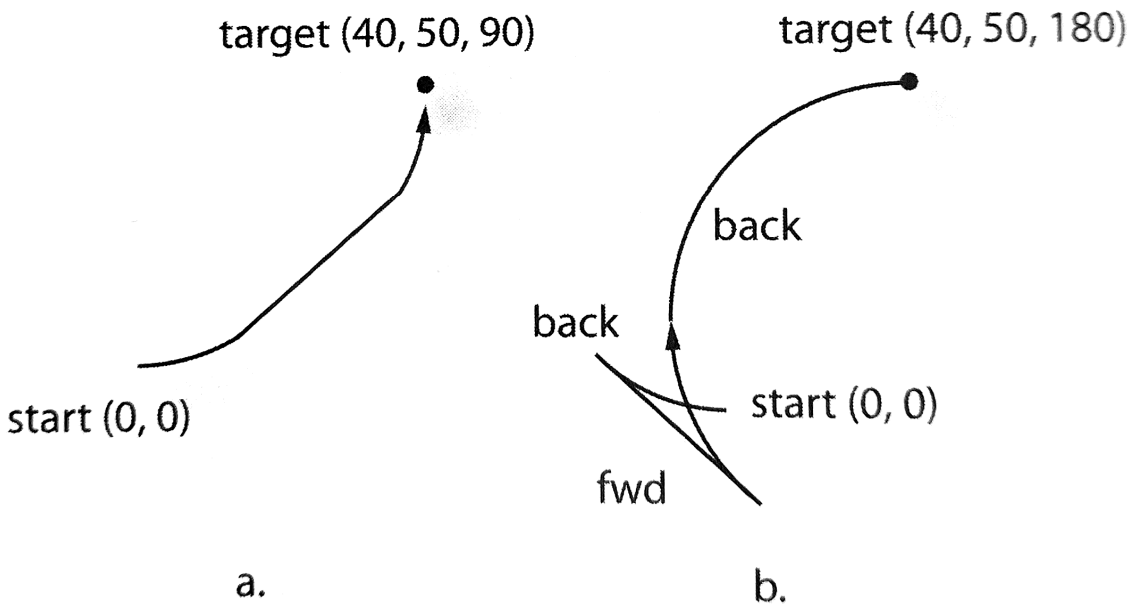
  Russell, Stuart, and Peter Norvig. "Artificial intelligence: a modern approach. Fourth Edition." (2020).

Examples in leJOS 1

- How to use the navigator
  - Steering.java
    - Page 369 in the textbook
    - Correct the code by yourselves
      - DifferentialPilot is deprecated: use MovePilot + Chassis
    -
```
Navigator nav = new Navigator (pilot);
nav.goTo(40, 50, 90);
nav.goTo(0, 0, 0);
nav.goTo(40, 50, 180);
```
    - The `Waypoint` can be very useful in navigation
    - Steering robot can't turning in place
      - Calculate a path using arcs and straight lines
      - Computational geometry is involved: not our focus

Examples in leJOS 2

- How to use the navigator
  - Steering.java (cont'd)
  - The heading direction in the pose is important



Examples in leJOS 3

- MoveListener
  - https://lejos.sourceforge.io/ev3/docs/lejos/robotics/navigation/MoveListener.html
  - An interface to report the moves made by the robot in real time
    - public void moveStarted(Move move, MoveProvider mp)
    - public void moveStopped(Move move, MoveProvider mp)
  - A MoveListener instance can be added to a MoveProvider (any pilot, e.g. MovePilot)
  - PerpetualListener.java
    - Page 356 in the textbook
    - Move until it encounters an obstacle, and then backs up and changes direction
    - Try to correct the code

Examples in leJOS 4

- Odometry calculation
  - OdometryPoseProvider
    https://lejos.sourceforge.io/ev3/docs/lejos/robotics/localization/OdometryPoseProvider.html
  - It requires a MoveProvider (e.g. MovePilot)
  - The MoveProvider updates the OdometryPoseProvider when a move is made
  - Odometry.java
    - Page 400 in the textbook
    - Try to correct the code
    - Result
      - ```
        X = -81cm
        y = 130cm
        heading = 180 degrees
        ```
      - But if Chassis is used: the unit will be mm
        ```

Examples in leJOS 5

- PathFinding (Optional)
  - https://lejos.sourceforge.io/ev3/docs/lejos/robotics/pathfinding/package-summary.html
  - https://github.com/mindstorms-cop/lejos-ev3/tree/master/ev3classes/src/lejos/robotics/pathfinding
  - AstarSearchAlgorithm::findPath(Node start, Node goal)
    - How to use Node to build the map is similar as what we have learned in Data Structures
  - For customized search algorithm, you can use the OccupancyGridMap
    - https://lejos.sourceforge.io/ev3/docs/lejos/robotics/mapping/OccupancyGridMap.html

Examples in leJOS 6

- How to build the map
  - LineMap could be very useful for the project
    - A map of a room or other closed environment, represented by line segments
  - https://lejos.sourceforge.io/ev3/docs/lejos/robotics/mapping/LineMap.html
  - LineMapTest.java